

Projet Programmation Système Avancée – Cluster de Nœuds : Ping et Synchronisation

Gouaïch Abdelkader

2025

Table des matières

TP – Cluster de Nœuds : Ping et Synchronisation	1
Objectifs pédagogiques	1
Cahier des charges	1
Exercices intermédiaires	2
Recommandations	3
Exécution attendue	3

TP – Cluster de Nœuds : Ping et Synchronisation

Durée estimée : 3 heures **Niveau** : L3 / M1 (Systèmes distribués / Réseaux avancés) **Prérequis** : sockets TCP, multithreading en C, structures de données simples

Objectifs pédagogiques

- Implémenter un cluster de nœuds capables de communiquer entre eux.
 - Mettre en place une infrastructure RPC de base en TCP.
 - Synchroniser plusieurs processus à l'aide d'un mécanisme de ping.
 - Préparer le terrain pour l'implémentation d'un protocole de consensus distribué (ex. RAFT).
-

Cahier des charges

Structure du projet

Le projet doit être structuré de la manière suivante :

```
step1_ping/  
├─ Makefile  
├─ rpc.h / rpc.c  
├─ config.h / config.c  
├─ node.h / node.c  
└─ main.c
```

Chaque composant a un rôle spécifique :

- `rpc.*` : gestion du serveur RPC (basé sur TCP) et de l'envoi de messages de type ping.
 - `config.*` : chargement d'un fichier de configuration contenant les informations des nœuds.
 - `node.*` : logique de gestion des nœuds, synchronisation et démarrage.
 - `main.c` : programme principal, point d'entrée du système.
-

Fonctionnalités à implémenter

1. Infrastructure RPC

- Serveur TCP multi-thread capable de recevoir des requêtes de type ping et de répondre.
- Client TCP capable d'envoyer une requête ping et de recevoir un accusé de réception pong.

2. Configuration

- Fichier texte décrivant les nœuds du cluster : identifiant, adresse IP, port.
- Chargement de ce fichier à l'exécution et remplissage d'un tableau de structures.

3. Comportement du nœud

- Chaque nœud démarre son serveur.
- Il envoie des requêtes ping à tous les autres nœuds.
- Il attend que tous les autres répondent avant de continuer.

4. Synchronisation

- Boucle de tentative de ping à tous les pairs jusqu'à obtention d'une réponse de chacun.
 - Lorsque tous les pairs répondent, le nœud affiche un message de synchronisation réussie.
-

Exercices intermédiaires

Exercice 1 – Création du fichier de configuration

Créez un fichier `nodes.conf` avec le contenu suivant :

```
0 127.0.0.1 9000
1 127.0.0.1 9001
2 127.0.0.1 9002
```

Écrivez une fonction permettant de lire ce fichier et de stocker les informations dans une structure de type :

```
typedef struct {
    int id;
    char host[64];
    int port;
} peer_t;
```

À tester : affichez la liste des pairs chargés depuis le fichier.

Exercice 2 – Serveur TCP simple

Écrivez une fonction qui démarre un serveur TCP écoutant sur un port donné. Lorsqu'un client se connecte, affichez "Nouvelle connexion" et fermez la connexion.

Extension : utilisez un thread pour gérer chaque client de manière concurrente.

Exercice 3 – Message ping

Définissez un protocole simple où :

- Un message ping est représenté par un octet de valeur 1.
- Une réponse pong est un octet de valeur 1.

Ajoutez dans votre serveur une logique :

- Lire un octet depuis la socket client.
 - Si c'est un ping, répondre avec un pong.
-

Exercice 4 – Client TCP

Écrivez une fonction qui se connecte à un serveur donné (IP + port), envoie un ping, attend une réponse pong, puis ferme la connexion.

À tester : faites dialoguer un client et un serveur lancés à la main.

Exercice 5 – Boucle de synchronisation

Dans le code principal du nœud :

- Après le démarrage du serveur, envoyez un ping à chaque pair (sauf soi-même).
 - Si tous répondent, affichez "Tous les pairs sont disponibles".
 - Sinon, réessayez toutes les secondes.
-

Recommandations

- Travaillez avec un terminal par nœud (3 instances).
 - Assurez-vous que les ports sont libres avant de lancer les serveurs.
 - Ajoutez des logs simples pour suivre les interactions.
-

Exécution attendue

```
./node 0 nodes.conf
./node 1 nodes.conf
./node 2 nodes.conf
```

Chaque nœud démarre, attend les autres, puis affiche :

Node 0: all peers responded. Proceeding.

Node 0: ready to start RAFT logic
